



Capstone Project – Java Database Programming

[Visit our website](#)

Introduction

In this task, you will be tasked with designing and implementing a database. You will also write the code needed to interact with the database.

Developer Portfolio

At the end of this project, you can be adding this program to your developer portfolio. Today most applications rely heavily on data! A prospective employer will want to see that you can persist and manipulate data. This project should make it clear to a potential employer that you can follow a development process to create a data-driven program that is debugged, tested, refactored and documented and that meets a client's specifications! Put extra time and effort into this project to make sure that it really showcases the skills you have acquired!

The Task at Hand

You have been asked to **create a project management system** for a small structural engineering firm called "**Poised**". Poised does the engineering needed to ensure the structural integrity of various buildings. They want you to create a Java program that they can use to keep track of the many projects on which they work.

Poised stores the following information for each project that they work on:

- Project number.
- Project name.
- What type of building is being designed? E.g. House, apartment block or store, etc.
- The physical address for the project.
- ERF number.
- The total fee being charged for the project.

- The total amount paid to date.
- Deadline for the project.
- The name, telephone number, email address and physical address of the architect for the project.
- The name, telephone number, email address and physical address of the contractor for the project.
- The name, telephone number, email address and physical address of the customer for the project.

Poised wants to be able to use your program to do the following:

- Capture information about new projects. If a project name is not provided when the information is captured, name the project using the surname of the customer. For example, a house being built by Mike Tyson would be called “House Tyson” and an apartment block owned by Jared Goldman would be called “Apartment Goldman”.
- Update information about existing projects. Information may need to be adjusted at different stages throughout the lifecycle of a project. For example, the deadline might change after a meeting with various stakeholders.
- Finalise existing projects. When a project is finalised, the following should happen:
 - The project should be marked as “finalised” in some way and the completion date should be added.
- See a list of projects that still need to be completed (have not been finalised).
- See a list of projects that are past the due date.
- Find and select a project by entering either the project number or project name.



Take note

A key focus of this project will be ensuring that your code is correct, well-formatted and readable. In this regard, make sure that you do the following before submitting your work:

1. Make sure that your code is readable. To ensure this, add comments to your code, use descriptive variable names and make good use of whitespace and indentation. See [this style guide](#) to see how classes and methods should be named and how your program should be formatted.
 2. Make sure that your code is as efficient as possible. How you choose to write code to create the solution to the specified problem is up to you. However, make sure that you write your code as efficiently as possible.
 3. Make sure that all output that your program provides to the user is easy to read and understand. Labelling all data that you output (whether in text files or to the screen) is essential to make the data your program produces more user-friendly.
-



Practical task 1

1. Design and create a **MySQL database** called PoisePMS. Assume that each project can only be assigned to one Structural Engineer. Each project will also only have one Project Manager, one Architect and one customer.
2. Submit the following:
 - An **ERD** that shows the relationships between the tables in your database.
 - **Screenshots** of your console that show how each table was created.
3. Add at least two rows of data to each table in the database. Submit screenshots of your console that show how data is added to the tables.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Practical task 2

1. Using the **JDBC**, write a **Java program** with the functionality to:
 - **Read** and **write** data about projects and people associated with projects from your database. Your program should not use any text files.
 - **Capture** information about new projects and add these to the database.
 - **Update** information about existing projects.
 - **Delete** data about projects and people associated with them.
 - Finalise existing projects - when finalised the project should be marked as “finalised” and the completion date should be added.
 - Find all projects that still need to be completed from the database.
 - Find all projects that are past the due date from the database.
 - Find and select a project by entering either the project number or project name.
2. Besides meeting the above criteria, you should also do the following:
 - Remove all errors from your code. Take extra care to detect and remove all logical and runtime errors.
 - Document your code. Adhere to the **style guide**.
 - Use Javadoc to generate API documentation from documentation comments for your program.
 - Follow the **guidelines** to create a Readme file for this project.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
